



Data Quality Violations Detector

Home Assignment

This assignment evaluates your ability to build a data processing pipeline that reads structured Excel files, detects data quality violations, and generates a clear, interactive HTML report.

1. Background

A healthcare organization processes weekly billing data across two primary document types: Funding Reports (detailed claim-level records) and Invoices (weekly financial summaries). Each week, multiple funding report files and one invoice file are generated and stored in a date-named folder.

Due to operational complexity, data quality issues occasionally arise - such as the same batch appearing in multiple weeks, or the same claim being submitted under different batches. Your task is to build a system that systematically identifies these violations.

2. Data Description

2.1 Directory Structure

You are provided with a data/ folder containing 122 weekly subfolders. Each folder is named with the date in YYYYMMDD format (the Monday of that week).

```
data/  
20240105/  
  blueNU79-Funding Report 20240105-1.xlsx  
  blueNU79-Funding Report 20240105-2.xlsx  
  blueNU79-Funding Report 20240105-3.xlsx  
  BS 01.05.24 Bill.xlsx  
20240112/  
  blueNU79-Funding Report 20240112-1.xlsx  
  blueNU79-Funding Report 20240112-2.xlsx  
  BS 01.12.24 Bill.xlsx  
...
```

2.2 Funding Report Files

Filename pattern: blueNU79-Funding Report {YYYYMMDD}-{N}.xlsx

These files have a tabular structure with the following characteristics:

- Row 1 contains column headers
- Data starts from row 2 onward (typically 50-150 rows per file)
- The Batch ID is stored ONCE in a single cell (row 2, column B) and applies to ALL data rows in that file

Key columns in funding reports:

Column	Type	Description
batch_id	String	Unique batch identifier (e.g. 200013)
claim_number	String	Claim ID (e.g. CLM101205)
service_date	Date	Date of service
gross_billable_charge	Float	Billed charge amount
total_payment_amount	Float	Actual payment amount
plan_type	String	Insurance plan (PPO/HMO/EP0)
provider	String	Provider company name

Important Note:

The batch_id is NOT repeated on every row. It appears once at the top of the data (row 2, column B). You must propagate it to all rows when parsing.

2.3 Invoice Files

Filename pattern: BS {MM.DD.YY} Bill.xlsx

Invoice files are NOT tabular. They use fixed cell positions to store financial summary data for the week. You must read specific cells by row and column index.

Cell positions for invoice data:

Field	Row	Column	Example Value
Invoice Date	3	F	2024-02-09
Invoice Number	3	G	INV-20240209
Carrier Claims	12	F	330452.93
TPA Claims	13	F	220301.95
Total Claims	14	G	550754.88
Carrier Fees (5%)	15	G	16522.65
Total Amount Due	16	G	567277.53

Edge Case - NEGATIVE BATCH variant:

Some invoices have "NEGATIVE BATCH" in row 16, column A. In this case, the Total Amount Due value is in row 17 (column G) instead of row 16.

3. Task Requirements

3.1 Objective

Build a Python application that:

- Reads and parses all Excel files from the data/ directory
- Aggregates funding report data into a unified dataset
- Detects data quality violations (described below)
- Generates a comprehensive, well-designed HTML report

3.2 Required Technology

- Python 3.10+
- Polars - for all data manipulation and aggregation (do NOT use pandas)
- openpyxl - for reading Excel files (especially fixed-position invoice data)
- HTML/CSS/JS - for the output report (no framework required)

3.3 Violations to Detect

A. Duplicate Batch ID Violations

A batch ID should appear in only one funding report file. If the same batch_id is found in funding reports from different weeks/files, it is a violation.

For each violation, report:

- The batch ID
- Number of files it appears in
- Total rows across all occurrences
- Total overpaid amount (estimated duplicate payment)
- List of source files where it appears
- Processed dates

B. Duplicate Claim ID Violations

A claim ID should appear in only one batch. If the same claim_number appears under different batch IDs, it indicates a duplicate claim submission.

For each violation, report:

- The claim ID
- Number of different batches it appears in
- List of batch IDs
- Total payment amount across all occurrences
- Source files

3.4 HTML Report Requirements

The generated report must include:

- Summary statistics: total violations count, total batches processed, total files
- A table of duplicate batch violations with all relevant details
- A table of duplicate claim violations with all relevant details
- Per-directory (week) comparison of funding totals vs invoice totals
- Ability to explore batch details - clicking a batch should show its claims
- Pagination for large tables
- Clean, professional visual design with good readability

The report should be a standalone HTML file (inline CSS/JS) that can be opened in any browser without a server.

4. Bonus (Optional)

Beyond the core requirements, you are encouraged to add any additional value you see fit. Some ideas:

- Visualizations - charts or graphs showing trends, distributions, or anomalies
- Additional violation types you identify in the data
- Deeper exploration features in the report (filtering, search, sorting)
- Any other insights or information that could help an analyst understand the data

There is no single correct answer here. Show us how you think about data quality problems and how you would present findings to a non-technical stakeholder.

5. Deliverables

Submit a Git repository containing:

- Source code - modular, readable Python with clear separation of concerns
- Generated HTML report - the output from running your code against the provided data
- README.md - with setup instructions, how to install dependencies, and how to run the application
- requirements.txt or pyproject.toml - listing all dependencies
- The data/ folder should NOT be committed to the repository (add to .gitignore)

5.1 Running the Application

Your application should be runnable with a simple command, for example:

```
python main.py --input-path ./data --output-path ./output
```

The output should include the HTML report file. Include clear instructions in your README for any additional setup steps.

6. Evaluation Criteria

Your submission will be evaluated on:

- Correctness - does it identify all violations accurately?

- Code quality - clean, modular, well-structured code
- Report quality - is the HTML report readable, informative, and well-designed?
- Data handling - proper use of Polars, handling of edge cases
- Documentation - clear README, easy to set up and run

Good luck!